

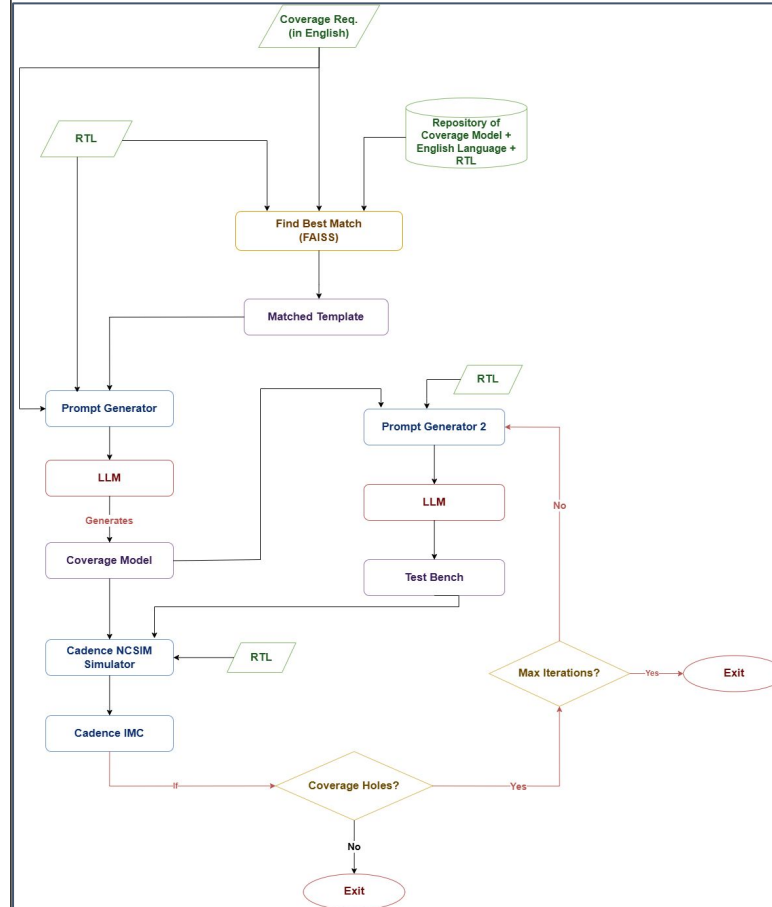
## Aim

The aim of this project is to develop an LLM-based framework that automates RTL verification by generating coverage models and testbenches, detecting coverage gaps, and improving coverage through iterative feedback.

## Literature Survey

- Traditional RTL verification uses **constrained-random testing** and **functional coverage**, but coverage closure is manual and time-consuming, as engineers must analyze coverage gaps and create new tests.
- EDA tools like **Cadence NCSim** and **IMC** enable simulation and coverage analysis, but do not automate test generation, leaving gap analysis and stimulus creation to designers.
- Template- and repository-based approaches** improve efficiency by **reusing past RTL-specification pairs**, guiding coverage model and testbench generation instead of starting from scratch.
- LLMs have shown strong capability in code and test generation (testbench), enabling automated coverage analysis and iterative test refinement, which forms the basis of this work.

## Proposed Framework



- The framework takes **RTL code** and a **human-readable coverage requirement** as input, then searches a repository of past examples to find the best matching **coverage model / English description / RTL template** pair.
- Using the matched template, the system generates two outputs through LLM prompts: a **formal coverage model** and a **testbench**. The framework supports different test-generation modes such as **randomized, constrained, and directed** iterations.
- The generated files are run in **Cadence NCSim/IMC** to collect coverage results. If coverage holes remain, the feedback is sent back to the LLM to regenerate the testbench, and this loop continues until the desired coverage is achieved or the maximum iterations are reached.

## Results

```
[bhavesh23169@dataools-server2 files6]$ python main.py --design adder8.v --regen_cov

#####
Coverage-Driven SV Verification Framework
Model      : gemini-2.5-flash-lite
Design     : adder8.v
Files      : interface.sv + coverage.sv + tb.sv + top.sv
Simulator  : irun
IMC        : imc
Threshold  : 90.0%
Max iters  : 5
Regen      : True
Run dir    : runs/run_20260420_163430

#####
[OK] Design: module='adder8'
[INFO] Inputs : ['a', 'b', 'cin']
[INFO] Outputs: ['sum', 'cout']
[INFO] Clock  : clk Reset: rst_n
[OK] Gemini client ready

#####
PHASE 1 -- Generating golden files
#####
[INFO] Generating interface ...
2026-04-20 16:34:30,139 [INFO] GeminiClient : Gemini model=gemini-2.5-flash-lite prompt=2575 chars
2026-04-20 16:34:31,714 [INFO] GeminiClient : finishReason=STOP in=750 out=178 tokens
[OK] interface: runs/run_20260420_163430/adder8_if.sv
[INFO] Generating coverage model ...
2026-04-20 16:34:31,721 [INFO] GeminiClient : Gemini model=gemini-2.5-flash-lite prompt=6385 chars
2026-04-20 16:34:35,361 [INFO] GeminiClient : finishReason=STOP in=1789 out=775 tokens
2026-04-20 16:34:35,362 [WARNING] GeminiClient : No code block found -- using raw response
[OK] coverage model: runs/run_20260420_163430/adder8_coverage.sv
[INFO] Generating top module ...
2026-04-20 16:34:35,363 [INFO] GeminiClient : Gemini model=gemini-2.5-flash-lite prompt=2029 chars
2026-04-20 16:34:36,401 [INFO] GeminiClient : finishReason=STOP in=596 out=151 tokens
[OK] top module: runs/run_20260420_163430/top_adder8.sv
[INFO] Golden files saved to run_dir (cached in design_dir for future runs)
```

Figure 1: RTL design being processed (Golden Files being generated).

```
covergroup cg_adder8 (ref logic [7:0] a, b, ref logic cin, cout);

  cp_a: coverpoint a {
    bins zero = {0};
    bins max = {255};
    bins mid = {[1:254]}; // not bins mid[] = {[1:254]}
  }

  cp_b: coverpoint b {
    bins zero = {0};
    bins max = {255};
    bins mid = {[1:254]};
  }

  cp_cin: coverpoint cin;

  // -----
  // 2. Behavior (overflow)
  // -----
  cp_overflow: coverpoint (a + b + cin > 255) {
    bins no_overflow = {0};
    bins overflow = {1};
  }

  // -----
  // 4. Input space coverage
  // -----
  cross_inputs: cross cp_a, cp_b, cp_cin;

  // -----
  // 5. Overflow context (only valid cases)
  // -----
  cross_overflow_inputs: cross cp_a, cp_b, cp_cin
    iff (a + b + cin > 255);

endgroup
```

Figure 2: Coverage model for RTL design.

## Pseudocode

### RTL COVERAGE AUTOMATION PIPELINE

#### INPUTS

```
RTL_code ← hardware RTL source file
coverage_req_english ← human-readable coverage spec
repository ← {coverage_model, english_desc, RTL} pairs (historical)
```

#### PHASE 1: TEMPLATE MATCHING

```
function find_best_template(coverage_req_english, repository):
  query_embedding = embed(coverage_req_english) // Facebook similarity / FAISS
  best_match = None
  best_score = -
  for each entry in repository:
    score = cosine_similarity(query_embedding, embed(entry.english_desc))
    if score > best_score:
      best_score = score
      best_match = entry
  return best_match // matched_template = {coverage_model, english, RTL}
matched_template = find_best_template(coverage_req_english, repository)
```

#### PHASE 2: COVERAGE MODEL GENERATION (Branch 1)

```
function generate_coverage_model(RTL_code, matched_template):
  prompt = prompt_gen_1
  (rtl = RTL_code,
   template = matched_template)
  coverage_model = LLM_1(prompt)
  return coverage_model
coverage_model = generate_coverage_model(RTL_code, matched_template)
```

#### PHASE 3: TESTBENCH GENERATION (Branch 2)

```
function generate_testbench(RTL_code, coverage_req_english, matched_template):
  prompt = prompt_gen_2
  (rtl = RTL_code,
   coverage_req_eng = coverage_req_english,
   template = matched_template)
  testbench = LLM_2(prompt)
  return testbench
testbench = generate_testbench(RTL_code, coverage_req_english, matched_template)
```

#### PHASE 4: SIMULATION & COVERAGE ANALYSIS

##### LOOP

```
MAX_ITERATIONS = N // configurable
iteration = 0
loop:
  if iteration >= MAX_ITERATIONS:
    log("Max iterations reached | exiting.")
    EXIT
  // Run simulation
  simulation_results = NCSIM(RTL_code, testbench,
    coverage_model)
  // Extract coverage metrics
  coverage_report = IMC(simulation_results)
  // Check for coverage holes
  holes = find_coverage_holes(coverage_report)
  if holes is empty:
    log("Full coverage achieved.")
    EXIT
  // Coverage holes found | regenerate testbench
  log("Iteration {iteration}: {len(holes)} coverage holes found.
  Re-generating...")
  testbench = generate_testbench
    (RTL_code,
     coverage_req_english,
     matched_template,
     feedback = holes // pass holes as additional context to Prompt
     Gen 2)
  iteration += 1
  continue loop
```

##### OUTPUTS

```
coverage_model → formal coverage specification
testbench → simulation testbench
coverage_report → final IMC coverage report (100% or max
  iters reached)
```

## Future Work

- Automate **coverage model and testbench generation** using the repository, reducing manual effort in creating verification components.
- Improve **scalability and reusability** so the framework can handle diverse RTL designs effectively.
- Develop stronger **iterative refinement and feedback-driven coverage closure**, enabling the system to automatically reach target coverage with minimal human intervention.

## References

- S. Saurabh, Introduction to VLSI Design Flow. Cambridge University Press, 2023.
- D. Yadav, P. Bhagat, P. Beniwal, and S. Saurabh, "CoDriVer: A Tool for Coverage-driven Functional Verification of RISC-V Processors," Indraprastha Institute of Information Technology, New Delhi, India.